

Statistical Inference and Modeling

2102575

Suwichaya Suwanwimolkul¹

¹Electrical Engineering
Chulalongkorn University

Semester II, 2025

- Lecture 1: Introduction
- Lecture 2: Model selection
- Lecture 3: Resampling
- Lecture 4: Linear regression
- Lecture 5: Robust regression
- Lecture 6: Classification
- Lecture 7: Logistic regression
- Lecture 8: Neural networks
- Lecture 9: K-nearest neighbor
- Lecture 10: Bayesian decision, LDA, QDA, and naive Bayes
- Lecture 11: Support vector machine
- Lecture 12: Tree-based methods
- Lecture 13: Bagging, random forest, boosting
- Lecture 14: Unsupervised learning and clustering algorithms
- Lecture 15: Unsupervised learning and PCA
- Lecture 16: Gaussian mixture model clustering
- Lecture 17: DBSCAN clustering

2. Model selection

2102575

Suwichaya Suwanwimolkul¹

¹Electrical Engineering
Chulalongkorn University

Semester II, 2025

- 1 Table of Contents
- 2 Flexibility vs Interpretability
- 3 Model components and why we need model selection?
- 4 How to select a model.
- 5 U-shape of generalization error
Observe over-fitting on validation performance
- 6 Cross Validation
Splitting Data
Example — Cross validation on polynomial order
Trade-off for k -fold
- 7 Model selection scores
Model selection scores
Examples
- 8 Details of each scores
Log-likelihood based scores (AIC, AICc, BIC)
Akaike's final prediction error (FPE)
Adjusted R^2
- 9 Exercises

Summary: bias and variance

From Lecture 1, mean square error (MSE) between $\hat{Y} = f(X)$ and the measurements Y can be used to analyze the bias and variance:

$$\text{MSE}(\hat{Y}) = \overset{\text{Reducible}}{\text{Bias}_{y_o}[f(X)]^2 + \text{Var}_X[f(X)]} + \sigma^2$$

where

- X is the set of input information (an independent variable).
- $\hat{Y} = f(X)$ is the prediction given the input information (this results in a dependent variable)
- **Reference:** y_o is the ground truth label.
- In **real settings**, instead of getting y_o , we may be able to access the measurements Y that is corrupted by noise: $Y = y_o + \mathcal{E}$. Usually, the measurement noise $\mathcal{E}$ is independent from X and has zero mean $\mathcal{E} \sim \mathcal{N}(0, \sigma^2)$.

*Generally, the best model is the one with low **bias** and low **variance**.*

Review: bias and variance

- Model with high flexibility $\Rightarrow$ often **high prediction accuracy**.
- Model with low flexibility $\Rightarrow$ often **high interpretability**.

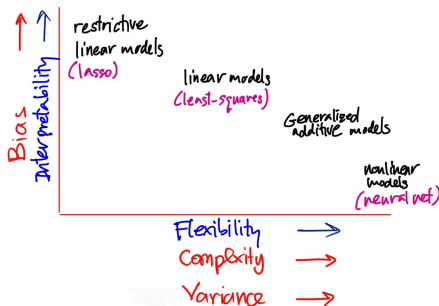


Figure: Trade-off between interpretability and accuracy [2, 3]

Model components and why we need model selection?

Data components in learning

- The input: X is the independent variables (predictors, input features)
- The output: $\hat{Y}$ is the dependent variables (response, prediction, predicting label, predicting target)
- The **reference**:
 - y_o is the noiseless reference (the ground truth label or the ground truth target).
 - Y is the noisy reference (the measurements).

Learning phase: Training, Validation, Testing

$$Y = y_o + \epsilon$$

Why we need model selection?

- ① **Quality of the model**: defined by a **measure of the goodness**, e.g., mean-squared error (MSE), residual sum of square (RSS), R^2 statistic (R^2)...
 - A more complex model **can have a lower bias** than the simpler one due to the higher flexibility to capture data.
 - But the more complex model **can have higher variance, if it is over-fitting to the training data.**
- ② **Price of the model**: computational **complexity**, which often depends on data dimension and algorithm complexity.
 - A more flexible model (more complex) also requires more computational power.

How to select a model

Goal: select a model that achieve a good balance between quality and computational complexity.
Types of model selection:

- 1 **Cross validation.** Select by choosing the model that gives the **best trade-off** between the computational **complexity** versus **model quality** on the **validation dataset** (unseen data). Many evaluation metrics can be used with this type of model selection.
- 2 **Model selection scores.** The selection scores consists of two mathematical terms: (a) model quality term and (b) model complexity term. The selection is done by finding the model that gives the best scores, which can be performed directly on the training dataset.


$$\text{Model Score} = \text{Quality} + \text{Complexity}.$$

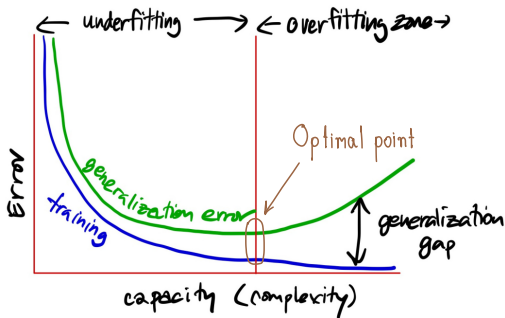
↓

model's price

U-shape of generalization error

Learning phase.

- **Training step:**
This step is used in finding θ and $f(\cdot; \theta)$.
 - **Validation step:**
This step aims at finding the hyperparameters of the trained model or performing **model selection**.
 - **Testing step:**
this happens during applying $f(\cdot; \theta)$ for deployment.
- × Either the error on the validation dataset and **testing dataset** (unseen data) can be used to represent the generalization error.



U-shape of generalization error

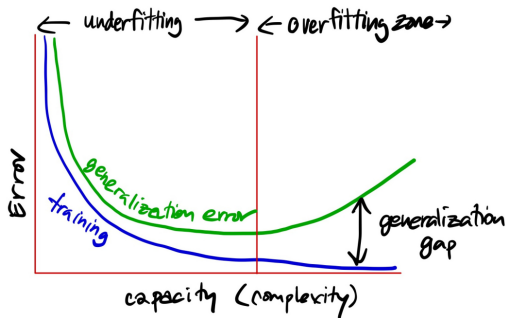


Figure: Generalization error [2, 3]

- Training errors always decrease as model complexity increase.
- Generalization error initially decreases as model picks up relevant features of data.
- However, if the model complexity exceeds a certain degree, the generalization error can rise up again — this is when we observe overfitting.

Observe overfitting on test error

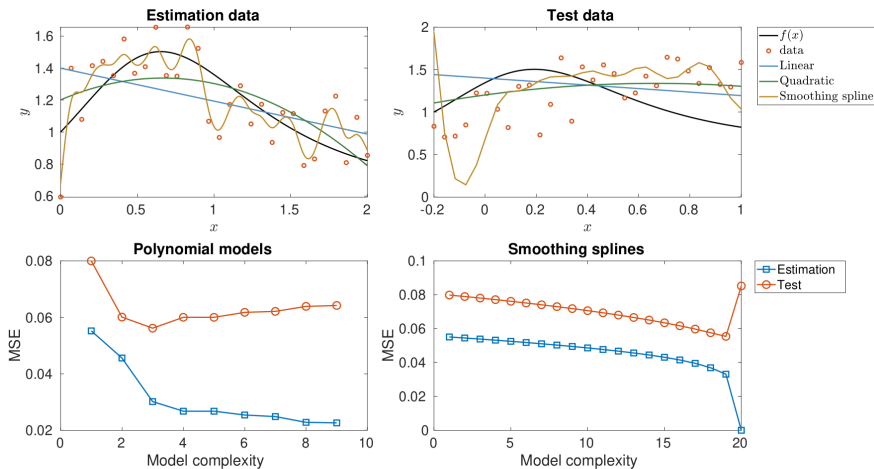


Figure: When overfitting happens: its impact on the training and test/validation error [3]

- Too complex models cannot generalize well on test/validation data.
- Overfitting occurs when MSE on test/validation set decreases but starts to rise again.

Cross Validation

* เพื่อให้งานตรงกัน*

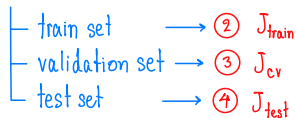
สมมติว่า $\hat{y} = \theta_0 + \theta_1 x + \theta_2 x^2 + \dots + \theta_d x^d$

Obj fⁿ : $\min_{\theta} \sum (y - \hat{y})^2 + \lambda \sum \theta_i^2 = J_{\text{train}}(\theta; \lambda, d)$

Regularization

Hyperparameter.

Data set



①. objective เรามี 3 set of parameter θ, λ, d

②. ในการเลือกโมเดลหนึ่งๆ เราจะ fix λ, d ไว้ค่าๆหนึ่งเหมือน θ
↳ โดยให้ J_{train} ต่ำสุด

③. เมื่อ J_{train} ต่ำสุด เราจะคำนวณ J_{cv} สำหรับทุก (λ, d)

④. The best model คือ J_{cv} ที่ต่ำที่สุด. ส่วน J_{test} ใช้ Report สายๆ

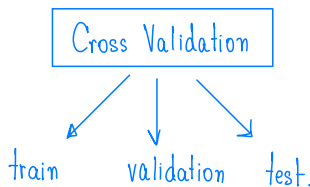
J_{train} สูง และ $J_{\text{cv}} \approx J_{\text{train}} \Rightarrow \text{Underfit}$
 J_{train} ต่ำ แต่ $J_{\text{cv}} \gg J_{\text{train}} \Rightarrow \text{Overfit}$
 J_{cv} ต่ำ $\Rightarrow \text{Good!}$

Cross Validation

Cross validation is a data splitting technique, where the data is split into training, validation, and testing dataset. The performance on validation dataset is used to estimate *the performance when testing the model (deployment)*.

Three approaches are often used to split the data.

- Validation set approach (hold-out)
- Leave-one-out cross validation
- k -fold cross validation



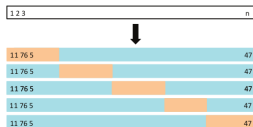
Splitting Data Assume ว่าเราแบ่ง Test set ออกไปแล้ว*

- **Training set**: used for fitting a model.
- **Validation set**: used for predicting the response from the fitted model.

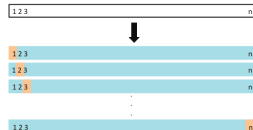
Hold Out



k -fold



Leave-one-out (LOOCV)



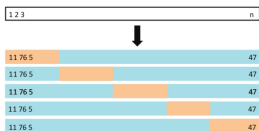
- **Hold Out**: randomly split data into two sets.
- **k -fold**: randomly split data into k folds; leave 1 fold for validation.
 - **repeat** k times where each time a different fold is regarded as validation set and compute $MSE_1, MSE_2, \dots, MSE_k$.
 - the test error rate is estimated by **averaging** the k MSE's.
- **Leave-one-out or LOOCV**: is actually a k -fold but with $k = m$ where m is the total number of sample or the total number of batches.

Splitting Data

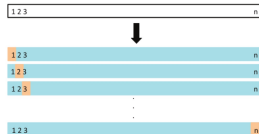
Hold Out



k-fold



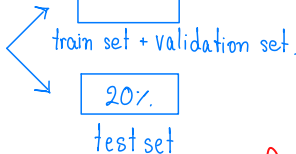
Leave-one-out (LOOCV)



ในชีวิตรจริง

100%
Raw Data

Split Data



Model Selection ตวาที่

Alt.

1. ทำ Split 60:20
2. ทำ k-fold
3. ทำ LOOCV

อย่างไร k-fold กับข้อมูลทั้งก้อน.

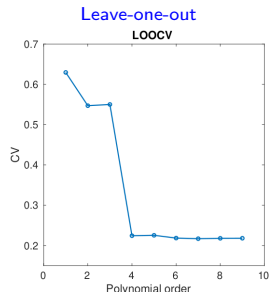
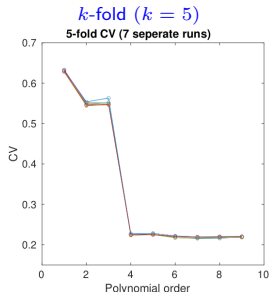
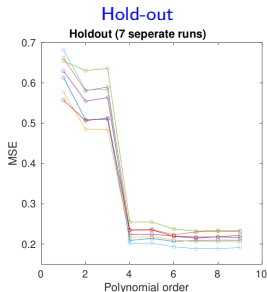
Splitting Data

```
def train_validate_test_split(df, train_percent=.6, validate_percent=.2, seed=None):  
    np.random.seed(seed)  
    perm = np.random.permutation(df.index)  
    m = len(df.index)  
    train_end = int(np.floor(train_percent * m))  
    validate_end = int(np.floor(validate_percent * m) + train_end)  
  
    train = df.iloc[perm[:train_end]]  
    validate = df.iloc[perm[train_end:validate_end]]  
    test = df.iloc[perm[validate_end:]]  
  
    train = train.drop(columns=["index"])  
    validate = validate.drop(columns=["index"])  
    test = test.drop(columns=["index"])  
  
    train = train.reset_index(drop=True)  
    validate = validate.reset_index(drop=True)  
    test = test.reset_index(drop=True)  
  
    return train, validate, test
```

60%	20%	20%
-----	-----	-----

Example — Cross validation on polynomial order

Using 500 training samples show 7 runs of holdout, k -fold, and LOOCV.



Results:

- **Hold-out** has high variation since it depends on random splitting.
- **k -fold ($k = 5$)** has less variation because MSE is averaged over 5 folds.
- **LOOCV** requires running $N = 500$ loops (high computational cost), but the MSE_i 's are highly correlated.

Trade-off for k -fold

Examine the bias and variance of test MSE...

Properties	Hold out	k -fold	LOOCV
Computation	low	feasible	high
Num. Samples	Ratio (e.g. 70:30)	$(k-1)m/k : m/k$	$m-1 : 1$
Stability	Highly unstable	Intermediate	Stable

- For k -fold and LOOCV, the validation MSE is calculated by taking the average of many MSE's.
- Most of MSE's from LOOCV are highly correlated
- However, MSE's of k -fold are less correlated (since LOOCV uses more overlapped data in training – hence, fitted models are almost identical)

7 Model selection scores

- Model selection scores
- Examples

Model selection criterion: main idea

ก่อนหน้านี้ เรารู้ว่าเราสามารถวัด MSE ในการเลือก Model ได้ โดย Plot MSE ระหว่าง Training และ Validation เพื่อดู Curve

Parsimony principle: among competing models which all explain the data well, the model with the smallest number of parameters should be chosen.

บน Training.

$$\text{Model selection score} = (1) \text{ loss function} + (2) \text{ model complexity}$$

- The first term (1) **loss function** is to assess the model quality, e.g., likelihood function, RSS, MSE, Fit Percent ($1 - \frac{\|y - \hat{y}\|}{\|y - \bar{y}\|} \times 100\%$).
- The second term (2) **model complexity** is to penalize the model order and grows as the number of parameters increases.
- The best model is the one with **the lowest score.**

deal with ①. model score คือสิ่งที่เราจะต้อง minimize.

underfit ②. loss function $\Rightarrow$ คุณภาพของโมเดล e.g. $Z(\theta; X)$, LSM, ...

overfit. ③. model complexity $\Rightarrow$ ความซับซ้อนของโมเดล e.g. Ridge, Lasso. $\Rightarrow$ การปรับโมเดล.

Model selection scores

Let's define ...

Log-likelihood ($\mathcal{L}$) and/or loss function (L) indicates **the model quality**;

Number of parameters (n) indicates **the model complexity**;

Number of data points (m);

Set of model and parameter α such as $\{f(\cdot), \theta\}$

Model selection scores:

- Akaike information criterion (AIC): $\text{AIC}(\alpha) = -2\mathcal{L}(\alpha) + 2n$
 - Corrected Akaike information (AICc): $\text{AICc}(\alpha) = -2\mathcal{L}(\alpha) + 2n + \frac{2n(n+1)}{m-n-1}$
 - Bayesian information criterion (BIC): $\text{BIC}(\alpha) = -2\mathcal{L}(\alpha) + n \log m$
 - Akaike's final prediction error (FPE): $\text{FPE}(\alpha) = V(\alpha) \left(\frac{1+n/m}{1-n/m} \right)$
 - Mallow's C_p : $C_p(\alpha) = \frac{1}{m} (\text{RSS}(\alpha) + 2n\hat{\sigma}^2)$
-
- Adjusted R^2 : $1 - \frac{\text{RSS}(\alpha)}{\text{TSS}(\alpha)} \cdot \frac{(m-1)}{(m-n-1)}$ } maximize
- Handwritten notes:*
- model quality:** points to $-2\mathcal{L}(\alpha)$ in the formulas.
 - complexity:** points to the terms involving n and m .
 - list:**
 - 1. Overfitting
 - 2. Computational Cost e.g. Time.
 - minimize** (with a bracket pointing to the first four formulas)
 - ปล. 1579: max $\mathcal{L}(\alpha)$.**

Note:

- $\text{RSS}(\alpha) := \sum_{j=1}^m (y_j - \hat{y}_j)^2$; each $\hat{y}_j$ is estimated value from the model and parameter defined in α .
- For AIC, AICc, BIC, FPE, C_p , the one with lower score is the better choice.
- For Adjusted R^2 , the one with higher score is the better choice.

Examples

Model function: $\hat{y} = \sum_{i=1}^n a_i \cos(ix) + b_i \sin(ix)$ for $n \in \{1, 20\}$.

- n is the number of basis function (number of parameters)
- The higher n ; the more complex model.
- Set the effective number of parameters $2n$ (num. of $a_i \cos(ix)$ and $b_i \sin(ix)$).
- Compute AIC, AICc, BIC, C_p and adjusted R^2 .

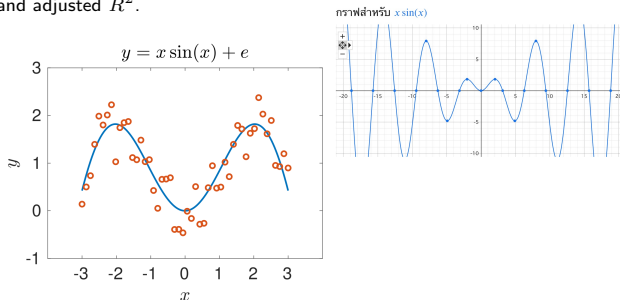
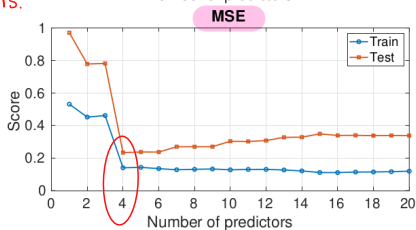
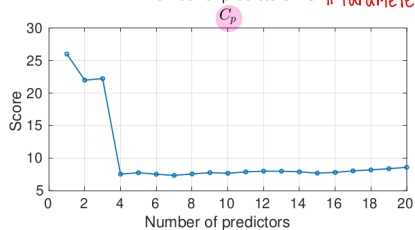
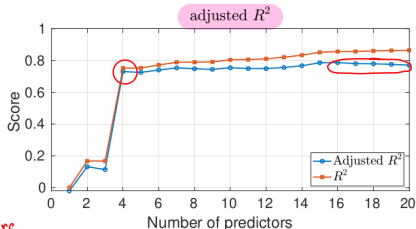
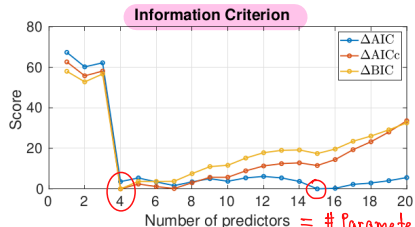


Figure: Example of the modeled datapoints [3]

- Blue line denotes the target function is $y = x \sin x + e$.
- Red dots are the predicted points using the model function

Examples

- AICc and BIC picked 4 basis, while C_p and AIC chose 7 basis function.
- Train MSE always decreases, as well as, R^2 always increases but the curves have a knee around $n = 4$.



Details of each scores

8 Details of each scores

Log-likelihood based scores (AIC, AICc, BIC)

Akaike's final prediction error (FPE)

Mallow's C_p

Adjusted R^2

Log-likelihood based scores (AIC, AICc, BIC)

AIC, AICc, BIC use negative log-likelihood to indicate model quality:

log-likelihood $\Rightarrow$ How to find ?

definition

$$\text{AIC}(\alpha) = -2\mathcal{L}(\alpha) + 2n$$

$$\text{AICc}(\alpha) = -2\mathcal{L}(\alpha) + 2n + \frac{2n(n+1)}{m-n-1}$$

$$\text{BIC}(\alpha) = -2\mathcal{L}(\alpha) + n \log m$$

α = set of parameter
 $= \{f(\cdot), \theta\}$

- Number of parameters (n) indicates **the model complexity**;
- Number of data points (m);
- Log-likelihood ($\mathcal{L}$) and/or loss function (L) indicates **the model quality**.

For example, $\mathcal{L} = \frac{1}{m} \sum_{j=1}^m (y_j - \mu)^2$. Notice that the log-likelihood ($\mathcal{L}$) and loss function (L) are also a function of the number of data points (m).

Properties

- When the number of samples is large, AIC can be the best criterion for choosing a good balance model.
- AICc(α) is the corrected version of AIC for small m ; AICc(α) penalizes more on complexity (quadratic in n).
- When the number of samples becomes large (large m), AICc(α) approaches AIC.

How to compute $\mathcal{L}(\alpha)$...

AIC is an approximation of **Kullback-Leibler (KL) divergence** between the true density $p(y|\alpha^*)$ and the model $p(y|\alpha)$ with **the estimated α** .

we assume y is Gaussian.

$$KL(p(y|\alpha^*), p(y|\alpha)) = \int_y \underbrace{p(y|\alpha^*)}_{\text{neglect.}} \log \frac{p(y|\alpha^*)}{p(y|\alpha)} = E_y \left[\log \frac{p(y|\alpha^*)}{p(y|\alpha)} \right]$$

State: assume,

$$Y|\alpha^* \sim \mathcal{N}(0, \sigma^2)$$

neglect. $\log p(y|\alpha^) - \log p(y|\alpha)$*

$$\boxed{\operatorname{argmin}_{\alpha}} KL(p(y|\alpha^*), p(y|\alpha)) = E_y \left[\log \frac{p(y|\alpha^*)}{p(y|\alpha)} \right] \equiv -E_y [\log p(y|\alpha)]$$

According to a strong law of large number (SLLN),

$$\lim_{m \rightarrow \infty} \frac{1}{m} \sum_{j=1}^m \log p(y_j|\alpha) = E_y [\log p(y|\alpha)]$$

in practical

$$\begin{aligned} \operatorname{argmin}_{\alpha} KL(p(y|\alpha^*), p(y|\alpha)) &= \operatorname{argmin}_{\alpha} - \lim_{m \rightarrow \infty} \frac{1}{m} \sum_{j=1}^m \log p(y_j|\alpha) \\ &= \operatorname{argmax}_{\alpha} \lim_{m \rightarrow \infty} \frac{1}{m} \sum_{j=1}^m \log p(y_j|\alpha) \\ &\equiv \operatorname{argmax}_{\alpha} \sum_{j=1}^m \log p(y_j|\alpha) = \mathcal{L}(\alpha) \end{aligned}$$

Because $\lim_{m \rightarrow \infty} \frac{1}{m} \sum_{j=1}^m \log p(y_j|\alpha) > \sum_{j=1}^m \log p(y_j|\alpha) = \mathcal{L}(\alpha)$.

How to compute $\mathcal{L}(\alpha)$...? $\rightarrow p(y_j, \alpha) = \frac{1}{\sqrt{2\pi\sigma^2}} \cdot \exp\left(-\frac{(y_j - E[y_j; \alpha])^2}{2\sigma^2}\right)$

Plug in the set of model and parameter α such as $\{f(\cdot), \theta\}$

$$\mathcal{L}(\alpha) = \sum_{j=1}^m \log p(y_j, \alpha) = -m \log \sqrt{2\pi\hat{\sigma}^2} - \frac{\sum_j (y_j - f(x_j, \theta))^2}{2\hat{\sigma}^2}$$

เปลี่ยนค่า. แบบ 2

Note that if $f(x_j, \theta)$ gives the best maximum likelihood; then, $\therefore y_j \sim \mathcal{N}(E[y_j; \alpha], \sigma^2)$
 $f(x_j, \theta) := E[y_j]$

As a result, the scale $\sum_j (y_j - E[y_j])^2$ will be the variance of that the measurements ...

$$\sum_j (y_j - E[y_j])^2 \approx m\hat{\sigma}^2 \quad \text{note } \hat{\sigma}^2 \approx \frac{1}{m} \sum_{j=1}^m (y_j - E[y_j])^2$$

$$\mathcal{L}(\alpha) = -\frac{m}{2} \log 2\pi\hat{\sigma}^2 - \underbrace{\left(\frac{1}{2} \frac{\text{RSS}}{\sigma^2}\right)}_{=m} = -\frac{m}{2} \log 2\pi\hat{\sigma}^2 - \frac{1}{2}m$$

If we plug this into AIC, then $\text{AIC} = -2\mathcal{L}(\alpha) + 2d = (m \log 2\pi\hat{\sigma}^2 + m) + 2n$

Comparing two models:

$$\begin{aligned} \Delta\text{AIC} &= (2n + m \log \hat{\sigma}^2 + m \log 2\pi + m) - (2n' + m \log \hat{\sigma}'^2 + m \log 2\pi + m) \\ &= 2(n - n') + m(\log \frac{\hat{\sigma}^2}{\hat{\sigma}'^2}) \Rightarrow \text{To compare 2 models.} \end{aligned}$$

Loss Complexity

If two models have the same n and n' , then $\log \frac{\hat{\sigma}^2}{\hat{\sigma}'^2}$ will be making the cut...

Log-likelihood based score (BIC)

BIC penalizes more on complexity than AIC.

$$\text{BIC}(\alpha) = -2\mathcal{L}(\alpha) + n \log m$$

AICs and BIC may not provide the same conclusion. Let α_1 be a set of model parameters that are more complex than α_1 . That is, $n_1 = n_2 + k$.

- If you use AIC,

$$\begin{aligned}\text{AIC}(\alpha_1) - \text{AIC}(\alpha_2) &= -\mathcal{L}(\alpha_1) + 2n_1 - (-\mathcal{L}(\alpha_2) + 2n_2) \\ &= \mathcal{L}(\alpha_2) - \mathcal{L}(\alpha_1) + 2k\end{aligned}$$

ดีกว่า แย่กว่า

$$\begin{aligned}\text{AIC}(\alpha_1) < \text{AIC}(\alpha_2) &\iff \text{AIC}(\alpha_1) - \text{AIC}(\alpha_2) < 0 \\ &\iff \mathcal{L}(\alpha_1) - \mathcal{L}(\alpha_2) > 2k\end{aligned}$$

AIC picks complex model, if $\mathcal{L}(\alpha_1) - \mathcal{L}(\alpha_2) > 2k$ (it's worthy to use complex model for better model quality).

- If you use BIC

$$\begin{aligned}\text{BIC}(\alpha_1) - \text{BIC}(\alpha_2) &= -\mathcal{L}(\alpha_1) + n_1 \log m - (-\mathcal{L}(\alpha_2) + n_2 \log m) \\ &= \mathcal{L}(\alpha_2) - \mathcal{L}(\alpha_1) + (n_2 + k) \log m - n_2 \log m\end{aligned}$$

ดีกว่า แย่กว่า

$$\begin{aligned}\text{BIC}(\alpha_1) < \text{BIC}(\alpha_2) &\iff \text{BIC}(\alpha_1) - \text{BIC}(\alpha_2) < 0 \\ &\iff \mathcal{L}(\alpha_1) - \mathcal{L}(\alpha_2) > k \log m\end{aligned}$$

BIC picks complex model, if $\mathcal{L}(\alpha_1) - \mathcal{L}(\alpha_2) > k \log m$ (it's worthy to use complex model for better model quality).

Akaike's final prediction (FPE)

A loss function denoted as $v(\alpha)$ is used in a prediction error method (e.g., det or trace of error covariance $v(\alpha) = \text{Tr}((\mathbf{y} - \hat{\mathbf{y}})(\mathbf{y} - \hat{\mathbf{y}})^T)$).

$$\text{FPE}(\alpha) = v(\alpha) \left(\frac{1 + n/m}{1 - n/m} \right)$$

- Model complexity is in multiplicative form (as compared to additive form in AIC, BIC)
- When model output is scalar, $V(\alpha)$ is simply MSE. FPE reduces to

$$\text{FPE} = \frac{1}{m} \sum_{j=1}^m \epsilon^2(j, \alpha) \cdot \frac{1 + n/m}{1 - n/m}$$

where $\epsilon^2(j, \alpha) = (y_j - \hat{y}_j)^2$ and $\hat{y}_j = f(x_j|\alpha)$.

Akaike's final prediction (FPE)

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}_{m \times 1}, \quad \hat{\mathbf{y}} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_m \end{bmatrix}_{m \times 1} \Rightarrow \mathbf{y} - \hat{\mathbf{y}} = \begin{bmatrix} y_1 - \hat{y}_1 \\ \vdots \\ y_m - \hat{y}_m \end{bmatrix}_{m \times 1}$$

$$\begin{aligned} (\mathbf{y} - \hat{\mathbf{y}})(\mathbf{y} - \hat{\mathbf{y}})^T &= \begin{bmatrix} y_1 - \hat{y}_1 \\ \vdots \\ y_m - \hat{y}_m \end{bmatrix}_{m \times 1} \cdot \begin{bmatrix} y_1 - \hat{y}_1 & y_2 - \hat{y}_2 & \cdots & y_m - \hat{y}_m \end{bmatrix}_{1 \times m} \\ &= \begin{bmatrix} (y_1 - \hat{y}_1)^2 & (y_1 - \hat{y}_1)(y_2 - \hat{y}_2) & \cdots & (y_1 - \hat{y}_1)(y_m - \hat{y}_m) \\ (y_2 - \hat{y}_2)(y_1 - \hat{y}_1) & (y_2 - \hat{y}_2)^2 & \cdots & (y_2 - \hat{y}_2)(y_m - \hat{y}_m) \\ \vdots & \vdots & \ddots & \vdots \\ (y_m - \hat{y}_m)(y_1 - \hat{y}_1) & (y_m - \hat{y}_m)(y_2 - \hat{y}_2) & \cdots & (y_m - \hat{y}_m)^2 \end{bmatrix} \end{aligned}$$

$$\therefore \text{Tr}((\mathbf{y} - \hat{\mathbf{y}})(\mathbf{y} - \hat{\mathbf{y}})^T) = \sum_{j=1}^m a_{jj} = \sum_{j=1}^m (y_j - \hat{y}_j)^2 \neq$$

C_p is mostly used in linear regression with d predictors and homoskedastic noise.

$$C_p = \frac{1}{m}[\text{RSS}(\alpha) + 2n\hat{\sigma}^2]$$

where $\text{RSS}(\alpha)$ is calculated based on the training data.

- C_p uses quadratic loss to measure model quality.
- $\hat{\sigma}^2$ is an estimate of noise variance using **full** model.
- RSS/m always decreases when n increases; penalty on complexity is put on $2n\hat{\sigma}^2$.
- Other form of C_p exists: $C_p = \text{RSS}/\hat{\sigma}^2 + 2n + m$.

Adjusted R^2

R^2 (coefficient of determination) is based on the decomposition:

$$\underbrace{\sum_{j=1}^m (y_j - \bar{y})^2}_{\text{TSS}} = \underbrace{\sum_{j=1}^m (y_j - \hat{y})^2}_{\text{RSS}} + \underbrace{\sum_{j=1}^m (\hat{y}_j - \bar{y})^2}_{\text{ESS}} + \underbrace{2 \sum_{j=1}^m (y_i - \hat{y}_j)(\hat{y}_j - \bar{y}_j)}_{\text{zero if model has a constant}}$$

R^2 is the proportion of the total variation in Y that can be linearly predicted by X .

$$R^2 = 1 - \frac{RSS}{TSS}, \quad \text{adjusted } R^2 = 1 - \frac{RSS(\alpha)/(m - n - 1)}{TSS/(m - 1)}$$

- For linear model, $0 \leq R^2 \leq 1$ and always increases for larger models.
- The presence of n penalizes the criterion for the number of predictor variables.
- Adjusted R^2 increases, if the added predictor variables decrease RSS enough to compensate for the increase in n .

- The summary usages of AIC, AICc, BIC, FPE, C_p , R^2 in <https://www.statsmodels.org/dev/gettingstarted.html>.

- [1] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*. Springer Series in Statistics. New York, NY, USA: Springer New York Inc., 2001.
- [2] Gareth James et al. *An Introduction to Statistical Learning: with Applications in R*. Springer, 2013. URL: <https://faculty.marshall.usc.edu/gareth-james/ISL/>.
- [3] Jitkomut Songsiri. “Statistical Learning”. In: in EE575 Random Processes for EE. Chulalongkorn University, 2022. Chap. 1. Introduction to Statistical Learning. URL: <http://jitkomut.eng.chula.ac.th/ee575.html>.
- [4] Jitkomut Songsiri. “Statistical Learning”. In: in EE575 Random Processes for EE. Chulalongkorn University, 2022. Chap. 5. Resampling. URL: <http://jitkomut.eng.chula.ac.th/ee575.html>.